

Lab 6: Arrays

Computer Programming () |

Duration: 2 hours (in-lab) | **Topic:** 1D and 2D arrays, traversal, accumulation, passing arrays to functions

Objectives

- Declare, initialize, and traverse 1D and 2D arrays.
- Pass arrays to functions and understand that the size is *not* carried with them.
- Implement basic array algorithms: search, min/max, reverse, frequency count.
- Use indices safely (bounds are 0 to **size** - 1).

Quick Reference

Declaration and initialization.

```
int a[5] = {1, 2, 3, 4, 5};
int b[10] = {0};           /* all zeros */
int m[3][4] = {{1,2,3,4},{5,6,7,8},{9,10,11,12}};
```

Bounds. For `a[N]` valid indices are `0..N-1`. Out-of-bounds access is *undefined behavior* – it may seem to work and still be wrong.

Passing to functions.

```
int sum(int a[], int n);    /* size must be passed separately */
```

Inside the function, `sizeof(a)` is *not* the array size – it is the size of a pointer.

Common idiom: max scan.

```
int max = a[0];
for (int i = 1; i < n; i++) if (a[i] > max) max = a[i];
```

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Read, Reverse, Print (25 min)

Read an integer N ($1 \leq N \leq 100$) and then N integers into an array. Print the array in reverse order on a single line.

In-Lab Exercise 2: Min, Max, Average (25 min)

Read an array of up to 50 doubles and compute the minimum, maximum, and average. Write three separate functions; do not duplicate the loop.

In-Lab Exercise 3: Linear Search (30 min)

Write `int find(int a[], int n, int key)` that returns the index of the first occurrence of `key` in the array, or `-1` if not present. Test from `main` with a key that is present and one that is not.

In-Lab Exercise 4: Matrix Transpose (40 min)

Read a 3×4 matrix of integers and print its 4×3 transpose. Use two nested `for` loops. Format each cell with field width 4 so columns align.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Indexing

What does this print? (Watch the bounds!)

```
#include <stdio.h>
int main(void) {
    int a[] = {10, 20, 30, 40, 50};
    int s = 0;
    for (int i = 1; i < 4; i++) s += a[i];
    printf("%d %d\n", s, a[0] + a[4]);
    return 0;
}
```

Trace & Predict 2: 2D traversal

Predict the printed output:

```
#include <stdio.h>
int main(void) {
    int m[2][3] = {{1,2,3},{4,5,6}};
    for (int j = 0; j < 3; j++) {
        for (int i = 0; i < 2; i++) printf("%d ", m[i][j]);
        printf("\n");
    }
    return 0;
}
```

Find the Bug 1: Off-by-one + uninitialized accumulator

List all bugs and write the corrected program:

```
#include <stdio.h>
int main(void) {
    int a[5] = {2, 4, 6, 8, 10};
    int sum;
    for (int i = 0; i <= 5; i++) sum += a[i];
    printf("sum = %d\n", sum);
    return 0;
}
```

Write Code on Paper 1: Count occurrences

Write a complete program that reads N integers into an array ($N \leq 100$), then reads a target value and prints how many times the target occurs in the array.

Write Code on Paper 2: Reverse in place

Write a function `void reverse(int a[], int n)` that reverses the array *in place* (no auxiliary array). Then write a `main` that demonstrates it.